ELEC-H505 Project
Advanced Digital Architecture
Project Suggestion

Navid Ladner
Contact: navid.ladner@ulb.be

April 2025

1 Context

You will design and test an accelerator using Chisel. Below we provide a few ideas for such accelerators but we leave you with the option to propose your own ideas. If you wish to develop an alternative idea, please send us an email with a concrete outline of the proposed accelerator. As Chipyard already implements a few accelerators, you can look at them to have an idea of how they implement it. The project can be done individually or in teams of two students, depending on the project (see section 4.)

The goal of this project is to first design an accelerator and evaluate its performance. Then, you will be able to connect it to a RISC-V CPU using the peripherals (or as a co-processor). Based on this design, you will test the correct functioning of the accelerator coupled to the CPU.

2 Requirements

- Design of a Chisel accelerator
- Simulation of the accelerator using testbenches & waveforms
- MMIO or RoCC (co-processor) connection to the processor (can be Rocket or BOOM)
- Simulation of the complete system (processor, accelerator, peripherals, ...) using different kind of tests. We expect you to document your tests and present them in your report.
- Using waveform and schematics, explain how the processor and the accelerator communicate.

- Compare the performance of the accelerator vs software implementation (with a single core). Discuss the potential impact on the PPA (you don't need to implement it, just be critical about the design.)
- Always consider security implications e.g, scenarios where distrusting applications would share the accelerator without revealing confidential or meta data to each other.

Completing the above tasks should not take you more than one week ($\approx$ 40h) of full-time work and is sufficient to pass the course, assuming reasonable quality of your implementation and report. For grades at the upper end of the scale, we recommend to address one of the following optional complications of the project.

- **Optional:** Security mitigation implementation. In addition to the security considerations we ask you above, we want you to implement the mitigations considered and evaluate their impact.
- **Optional:** Prototype and evaluate your design on FPGA.
- **Optional:** Physical implementation of your design using OpenRoad
- **Optional:** Any additional feature, ideas of analysis and/or implementation, ...

You are welcome to propose different complications; if so, please discuss this with the teaching assistants or lecturers upfront.

3 Deadlines & Project Defense

3.1 Project Update

To make sure that you are on the right track for the project, we ask you to send us a project update report or to book a meeting so that we can discuss your progress. Prepare a small description of the accelerator and its application (1-2 page max.)

- If you intend to send a report, it must be sent **before Monday 05/05**.
- If you intend to book a meeting, this will be take place in the **week of 05/05**.

3.2 Final Report

The final report must be submitted on the UV **by Monday 26/05/2025**. The report must address all the requirements listed under 2.

3.3 Defense

The project defense will be held as an oral exam during the exams period, after 02/06.2025. In a presentation of about 10 minutes, you are expected to explain each step of the project, specifically the functioning and implementation of your accelerator, and how you could improve it. In the questioning part of the exam, we will discuss your design and link your work to concepts developed across the lectures and exercises of the course. You may prepare a few slides or bring your project report to the exam.

4 Project Ideas

4.1 Neural Network Accelerator

This project idea aims at developing processor support to accelerate the operations required for neural network computations. As this kind of application is highly data driven, computations within a general purpose processor are slow and power consuming. To reach fast computation (for self driving car, virtual reality, ...), accelerators such as **systolic arrays** are commonly used.

For this project, you will design an accelerator depending on the specific kind of neural network you want to accelerate. We recommend you to focus on Convolutional Neural Network (CNN) using systolic arrays. Some references of systolic arrays can be found on the UV page. You can choose to focus on **training** or **inference** or **both**.

Some References

- C source code for different types of neural network with a dataset. For CNN acceleration, you can look at cnn.c and mnist.c. In this repository, you also have some source codes you can use for testing.
- HardwareInformation FlowTracking
- Systolic Array Based Convolutional Neural Network Inference on FPGA
- A High-Performance Systolic Array Accelerator Dedicated for CNN

We recommend a team of 2 students for this project. Security aspect must be discussed but no implementations are required.

4.2 Image or Signal Processing

Currently, a lot of accelerators focuses on signal or image processing. For this project, we expect you to build one type of accelerator to do at least 2 of the following features:

- Image compressor (algorithm of your choice).
- Stenography: hide a picture within a picture and retrieve it.
- Convolution product.
- On-demand image dithering for a configurable target image size and color scheme.

Some References

- HardwareInformation FlowTracking

We recommend a team of 1 student for this project. Security aspect must be discussed but no implementations are expected.

In case you want to work on this project in a group of up to two students, you have to take the security aspect into account and implement mitigations to potential attacks.

4.3 Securely Multiplex Access to an MMIO Peripheral by Using Physical Memory Protection (PMP)

Imagine you have an MMIO peripheral that requires software to follow a well-timed access protocol (e.g., request resource, wait until resource is ready, use resource) but that needs to be securely shared between different applications without interference [6]. You would normally want to implement this through a driver that requires special protection and multiplexes access to the applications. RISC-V Physical Memory Protection (PMP) allows you to secure peripherals by defining MMIO regions and assigning access permissions to specific privilege modes, allowing you to implement the required control even in M-Mode for an embedded Operating System without S/U-Mode support, or for full OSs by denying all access to S/U modes [4]. In this project you will develop a demonstrator for this technique on BOOM or Rocket, with integration for either an embedded Real-Time OS (such as Zephyr RTOS) or Linux, and access the resulting security and availability guarantees.

Literature:

- Jo Van Bulck, Job Noorman, Jan Tobias Mühlberg, and Frank Piessens. Secure resource sharing for embedded protected module architectures. In *IFIP International Conference on Information Security Theory and Practice*, pages 71–87. Springer, 2015
- Gregor Haas, Micheal Chung, and David Kohlbrenner. Enabling confidential i/o on risc-v without hardware modifications. In *Proceedings of the 7th Joint Workshop on CPS&IoT Security and Privacy*, pages 19–33, 2025

4.4 Cache Partitioning, e.g., based on Physical Memory Protection (PMP) Configuration

Cache side-channels attack techniques exploit page sharing as a weakness that exposes processes to information leaks. For example, Yarom et al. [8] presented the FLUSH+RELOAD attack to monitor access to memory lines in shared pages. Unlike previous cache side-channel attacks, FLUSH+RELOAD targets the Last-Level Cache, which is shared across processor cores. Thus, attack program and the victim do not need to share the execution core. As Austa et al. [1] show, similar attacks are feasible on RISC-V.

In this project you will implement a defense mechanism against cache side channels: cache partitioning. If the cache is no longer shared between different tasks, these tasks are no longer vulnerable to cache side-channel attacks ([5, 2]). RISC-V comes with a Physical Memory Protection (PMP) Unit that can implement M-Mode isolation in main memory. Your task is to replicate and evaluate this mechanism for the cache.

Literature:

- Yuval Yarom and Katrina Falkner. {FLUSH+ RELOAD}: A high resolution, low noise, l3 cache {Side-Channel} attack. In *23rd USENIX security symposium (USENIX security 14)*, pages 719–732, 2014
- Cédrick Austa, Jan Tobias Mühlberg, and Jean-Michel Dricot. Systematic assessment of cache timing vulnerabilities on risc-v processors. *arXiv preprint arXiv:2510.08272*, 2025
- Dan Page. Partitioned cache architecture as a side-channel defence mechanism. *Cryptology ePrint Archive*, 2005

4.5 Interrupt Side Channel Exploitation and/or Mitigation

Interrupt side channels exploit basic behavior of microprocessors to enable an attacker process to observe the execution of a victim: On many processors, the latency between the arrival of an interrupt and the invocation of the interrupt service routine is determined by the execution time of the instruction that is being executed when the interrupt arrives. Given an attacker who is capable of scheduling interrupts (e.g., through a timer), the processor's behavior may allow an attacker to observe victim latencies at instruction level. Van Bulck et al. [7] show that this attack can be used to observe secret information from Trusted Execution contexts, and [3] et al. illustrate that the attack is feasible on RISC-V.

In this project you will develop a demonstrator for an interrupt side channel on Rocket or BOOM. You are free construct a vulnerable victim program as you see fit. Possible extensions could be to implement the attack with a "remote"

attacker who is using I/O interrupts, or explore a defense mechanism to harden to harden the victim programme against the attack.

Literature:

- Jo Van Bulck, Frank Piessens, and Raoul Strackx. Nemesis: Studying microarchitectural timing leaks in rudimentary cpu interrupt logic. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 178–195, 2018
- Johannes Geier, Lukas Auer, Daniel Mueller-Gritschneder, Uzair Sharif, and Ulf Schlichtmann. Compasec: a compiler-assisted security countermeasure to address instruction skip fault attacks on risc-v. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference*, pages 676–682, 2023

4.6 Original Ideas

If you have an idea of accelerator that you want to implement, we expect you to send us a description of the accelerator and a bullet-point list of your objectives for the project by email. We will then discuss the proposal with you during the labs.

References

[1] Cédrick Austa, Jan Tobias Mühlberg, and Jean-Michel Dricot. Systematic assessment of cache timing vulnerabilities on risc-v processors. *arXiv preprint arXiv:2510.08272*, 2025.

[2] Nicolas Gaudin, Pascal Cotret, Guy Gogniat, and Vianney Lapotre. A fine-grained dynamic partitioning against cache-based timing attacks via cache locking. In *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pages 173–179. IEEE, 2024.

[3] Johannes Geier, Lukas Auer, Daniel Mueller-Gritschneder, Uzair Sharif, and Ulf Schlichtmann. Compasec: a compiler-assisted security countermeasure to address instruction skip fault attacks on risc-v. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference*, pages 676–682, 2023.

[4] Gregor Haas, Micheal Chung, and David Kohlbrenner. Enabling confidential i/o on risc-v without hardware modifications. In *Proceedings of the 7th Joint Workshop on CPS&IoT Security and Privacy*, pages 19–33, 2025.

[5] Dan Page. Partitioned cache architecture as a side-channel defence mechanism. *Cryptology ePrint Archive*, 2005.

[6] Jo Van Bulck, Job Noorman, Jan Tobias Mühlberg, and Frank Piessens. Secure resource sharing for embedded protected module architectures. In *IFIP International Conference on Information Security Theory and Practice*, pages 71–87. Springer, 2015.

[7] Jo Van Bulck, Frank Piessens, and Raoul Strackx. Nemesis: Studying microarchitectural timing leaks in rudimentary cpu interrupt logic. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 178–195, 2018.

[8] Yuval Yarom and Katrina Falkner. {FLUSH+ RELOAD}: A high resolution, low noise, l3 cache {Side-Channel} attack. In *23rd USENIX security symposium (USENIX security 14)*, pages 719–732, 2014.